

CORSO DI BIG DATA — TOPIC 7: IOT STREAM ANALYTICS

Self Healing Fleet

Ingestion continua, detection real-time e analisi predittiva offline
su una flotta di AGV in un magazzino

Autore: Marcello Tascioni

Data: 21 luglio 2026

Repository: self-healing-fleet

Indice

1. Obiettivi del progetto

- 1.1 Contesto e vincoli del corso
- 1.2 Obiettivi funzionali
- 1.3 Un progetto di Big Data, non di robotica

2. Architettura e tecnologie utilizzate

- 2.1 Vista d'insieme: architettura lambda
- 2.2 Sorgente dati: ROS1 Noetic + Gazebo + TurtleBot3
- 2.3 Contratto dati: schema del messaggio di telemetria
- 2.4 Ingestion: Apache Kafka
- 2.5 Generatore sintetico per lo sweep di scalabilità
- 2.6 Speed layer: detection real-time in Spark Structured Streaming
- 2.7 Batch layer: persistenza e soglie adattive
- 2.8 Analisi predittiva offline (time series)
- 2.9 Serving layer: Spark SQL e sistema TAG con Hugging Face
- 2.10 Presentazione: backend Node.js e dashboard
- 2.11 Estensione: self healing sulla flotta reale
- 2.12 Orchestrazione e deployment

3. Risultati

- 3.1 Effectiveness: precisione della detection, della previsione e del TAG
- 3.2 Efficiency: scalabilità e latenza
- 3.3 Verifica end-to-end sulla flotta reale
- 3.4 Copertura dei requisiti del corso

4. Osservazioni conclusive

- 4.1 Lezioni tecniche
- 4.2 Limiti dichiarati
- 4.3 Sviluppi futuri

1. Obiettivi del progetto

1.1 Contesto e vincoli del corso

Il progetto è stato sviluppato per il corso di Big Data, in relazione al **Topic 7 — IoT Stream Analytics**. Il tema scelto è la gestione di una flotta di AGV (Automated Guided Vehicle) in un magazzino: i veicoli percorrono una mappa a grafo trasportando merci, inviando in continuo la propria telemetria (posizione, velocità, stato dei task, canali di salute del motore/batteria). Il sistema deve ingerire questo flusso continuo,

rilevarne le anomalie in tempo reale, prevederne i guasti su base storica, e rispondere a interrogazioni in linguaggio naturale.

Il vincolo di fondo, esplicitato fin dall'inizio nella documentazione di progetto, è che si tratta di **un progetto di Big Data, non di robotica**: il robot è unicamente la sorgente del flusso dati. Le tre "V" classiche sono tutte presenti nel disegno del sistema — *Volume* (il generatore sintetico permette di scalare da poche unità a migliaia di sorgenti simulate), *Velocity* (ingestion e detection in streaming, non batch), *Veracity* (dati fisicamente realistici, con rumore e vincoli reali imposti da un simulatore fisico, non generati "a tavolino") — a cui si aggiunge *Value*, dato dal layer di analisi predittiva che trasforma lo storico grezzo in previsioni azionabili.

1.2 Obiettivi funzionali

1. **Ingestion continua e realistica.** La sorgente dati doveva essere una vera pipeline ROS + Gazebo + TurtleBot3, non un generatore puramente sintetico fin dal principio — requisito esplicito del progetto per garantire dati con vincoli fisici reali (odometria, lidar, dinamica del robot), non solo formalmente conformi allo schema.
2. **Detection in tempo reale** di due categorie di anomalie: anomalie di *salute* (derive termiche, picchi di corrente, collasso della batteria, sensori bloccati) e anomalie *comportamentali* (deadlock e livelock su corridoi a corsia singola).
3. **Analisi predittiva offline** sullo storico dei canali di salute, per stimare *quando* un robot supererà una soglia critica (tempo di anticipo, o *lead time*), non solo *se* è già in anomalia.
4. **Riduzione dei falsi positivi** tramite un meccanismo di soglie adattive, calibrate sullo storico reale usando i guasti iniettati come ground truth.
5. **Interrogazione in linguaggio naturale** dello storico accumulato, tramite un layer text-to-SQL (TAG, Table-Augmented Generation) basato su un LLM open-source.
6. **Quattro tecnologie obbligatorie**, tutte presenti almeno in versione minima: Apache Kafka, Spark Structured Streaming, un modello di previsione su serie storiche, un LLM.
7. **Valutazione sperimentale** quantitativa, sia di *effectiveness* (precisione della detection e delle previsioni, accuratezza del TAG) sia di *efficiency* (scalabilità del throughput, latenza end-to-end), con un meccanismo di ground truth (``injected_faults``) che rende le metriche calcolabili in modo oggettivo, non stimato.

1.3 Un progetto di Big Data, non di robotica

Questo vincolo ha guidato diverse scelte architetturali documentate nel seguito: la navigazione dei robot non usa SLAM/mappa statica (irrilevante per gli obiettivi del progetto, il grafo del magazzino è già una roadmap nota); i canali di salute (temperatura motore, corrente, percentuale batteria) non esistono nel TurtleBot3 simulato e vengono *sintetizzati* nel nodo-ponte ROS → Kafka a partire da parametri nominali configurabili; l'iniezione dei guasti avviene sommando una firma parametrica alla telemetria realistica, non generando dati fittizi da zero. In un'estensione realizzata oltre il piano originale (§2.11), è stata anche introdotta un'unica, dichiarata eccezione a questo principio: per la sola flotta reale, un'anomalia di salute rilevata chiude l'anello

con un'azione minima (invio verso un nodo di riparazione) — la decisione applicativa resta comunque nel backend, non nel robot.

2. Architettura e tecnologie utilizzate

2.1 Vista d'insieme: architettura lambda

Il sistema è organizzato come una **architettura lambda**: gli stessi dati grezzi (il topic Kafka `telemetry`) alimentano sia un percorso *speed* a bassa latenza per la detection in tempo reale, sia un percorso *batch* più lento ma completo per l'analisi storica e predittiva. Un *serving layer* espone entrambi i risultati, sia alla dashboard live sia alle interrogazioni ad-hoc in linguaggio naturale.

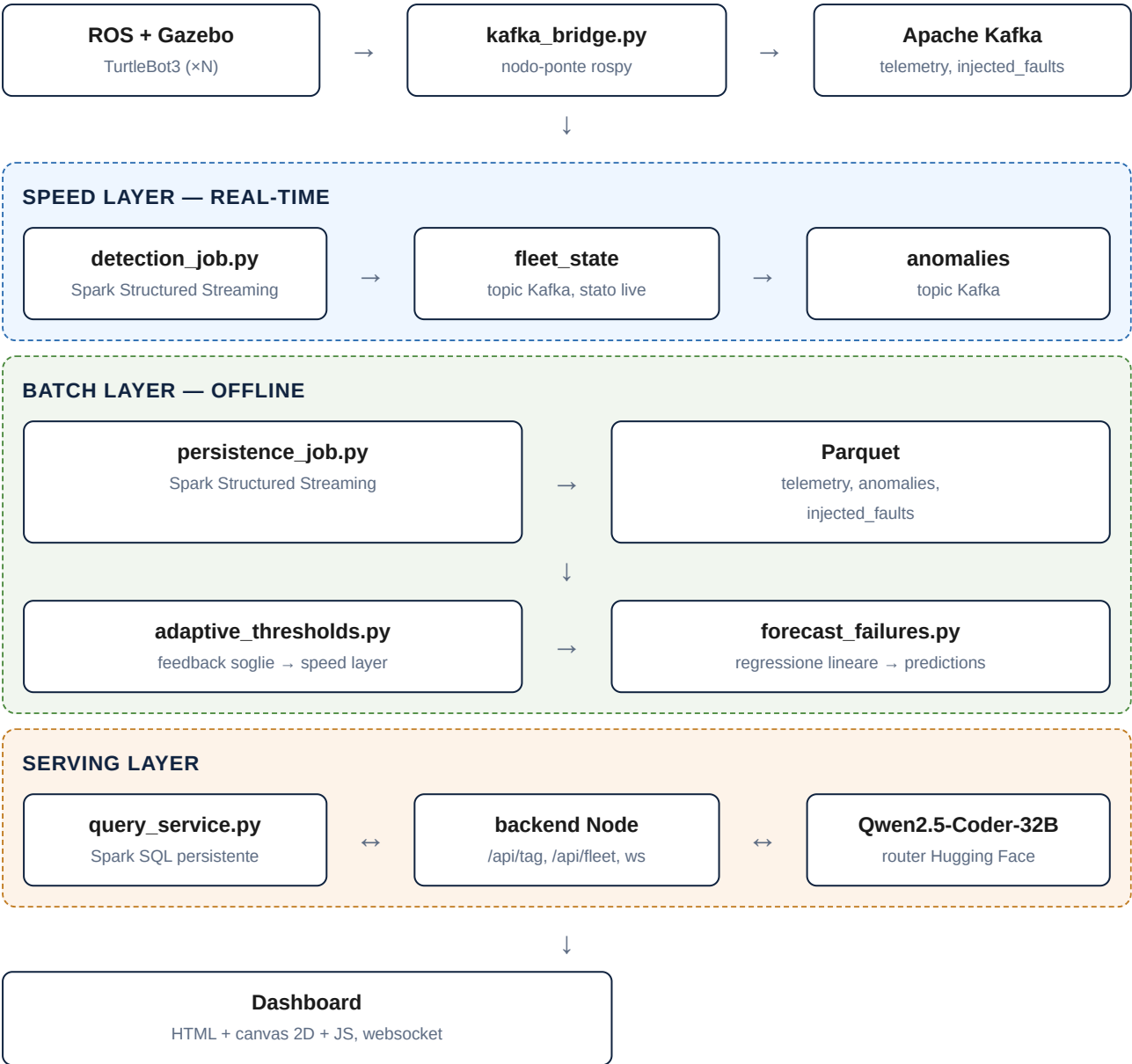


Fig. 1 — Architettura lambda del sistema: gli stessi dati grezzi alimentano il percorso real-time (speed layer) e quello storico (batch layer); il serving layer li espone entrambi alla dashboard e alle interrogazioni in linguaggio naturale.

2.2 Sorgente dati: ROS1 Noetic + Gazebo + TurtleBot3

La sorgente dati è una simulazione fisica, non un generatore di numeri casuali conforme a uno schema. Ogni robot è un TurtleBot3 "burger" simulato in Gazebo, controllato tramite lo stack di navigazione ROS standard (`move_base` con pianificatore locale DWA). La navigazione **non usa SLAM né una mappa statica/AMCL**: il costmap globale vive nel frame `odom` (60×40 m, nessun `static_layer`) — scelta deliberata, perché il grafo del magazzino (§2.3) è già una roadmap nota a priori, e introdurre SLAM avrebbe aggiunto complessità robotica irrilevante per gli obiettivi di Big Data del progetto. Ogni robot riceve in sequenza i nodi del proprio percorso come goal `actionlib`.

Multi-robot e scenari di conflitto

TurtleBot3 non supporta nativamente il multi-robot: è stato necessario uno script (`gen_robot_description.sh`) che inietta il namespace nei plugin Gazebo e rinomina esplicitamente i frame TF (a differenza dei topic, `tf2` non prefissa automaticamente i `frame_id` in base al namespace di lancio). La flotta di riferimento comprende 3 robot attivi (R1-R3) più un quarto robot di riserva (R4, §2.11); uno scenario esteso ("large") raddoppia la flotta attiva a 6 robot con due riserve (R4, R8), selezionabile con un parametro `scale` del file di lancio ROS.

Sintesi dei canali di salute e fault injection

Il nodo-ponte `kafka_bridge.py` sottoscrive `/odom`, `/cmd_vel`, `/scan`, `/move_base/goal`, `/move_base/result`. I tre canali di salute (`battery_pct`, `motor_current`, `motor_temp`) — assenti dal TurtleBot3 simulato — vengono sintetizzati a partire da parametri nominali condivisi (`health_channels_nominal` in `config/experiment.json`): la batteria si scarica in funzione dello stato di moto/idle e si ricarica in "charging", corrente e temperatura oscillano attorno a un valore nominale con rumore gaussiano. La posizione sull'arco corrente del grafo (`current_edge`) è calcolata proiettando (`x`, `y`) reali sul grafo.

L'iniezione dei guasti è realizzata da una classe `FaultInjector` che legge un `fault_schedule` (§2.3) e **somma una firma parametrica alla telemetria realistica prima della pubblicazione**, non genera un canale separato: un guasto iniettato attraversa esattamente lo stesso percorso dati di un guasto "vero". Le firme implementate sono quattro, tutte parametriche:

Firma	Canale	Meccanismo
<code>deriva_termica</code>	<code>motor_temp</code>	rampa lineare fino a un plateau
<code>spike_corrente</code>	<code>motor_current</code>	picco che resta elevato per la durata del guasto
<code>batteria_collasso</code>	<code>battery_pct</code>	discesa più ripida del drain rate nominale
<code>sensore_bloccato</code>	configurabile	valore congelato all'ultimo campione valido

A queste si affiancano due **scenari comportamentali** (deadlock, livelock), realizzati non come firme sui dati ma come vera e propria contesa di traffico: il grafo del magazzino include due archi a corsia singola (`capacity=1`, C-F e C-H). In un'estensione realizzata dopo il completamento del piano originale (§2.11), questi due archi sono stati fisicamente ristretti anche in Gazebo (pareti statiche SDF, corridoio libero di 0.8

m), così che i conflitti di traffico siano osservabili come vincoli fisici reali e non solo come un numero nel file di configurazione.

2.3 Contratto dati: schema del messaggio di telemetria

Lo schema del messaggio di telemetria è la fonte di verità unica dell'intero progetto: ogni componente (ponte ROS, generatore sintetico, job Spark, storage, layer TAG) si conforma ad esso. Un tick per robot, pubblicato in formato JSON sul topic Kafka `telemetry`:

```
{
  "ts": 1721400000000,
  "robot_id": "R3",
  "x": 12.4, "y": 3.1, "theta": 1.57,
  "v_lin": 0.22, "v_ang": 0.0,
  "cmd_v_lin": 0.25, "cmd_v_ang": 0.0,
  "battery_pct": 74.0, "motor_current": 1.8, "motor_temp": 41.5,
  "min_obstacle_dist": 0.9,
  "task_state": "moving",
  "current_edge": "C-F",
  "goal_node": "H"
}
```

`task_state` assume uno fra `idle`, `moving`, `blocked`, `charging` — il valore `blocked` richiede che *sia* la velocità lineare *sia* quella angolare restino sotto una soglia epsilon per oltre 5 secondi con un goal attivo (una rotazione pura di recovery, tipica di `move_base`, resta quindi classificata `moving`, non `blocked` — un dettaglio rilevante per il rilevatore di livelock, §2.6). Posa, velocità e `min_obstacle_dist` provengono da Gazebo (odometria e lidar reali); i canali di salute sono sintetizzati come descritto sopra.

La mappa del magazzino (`config/warehouse_graph.json`) è un grafo con nodi (id, coordinate, tipo) e archi (id, estremi, capacità, lunghezza); i robot la seguono come roadmap, e la posizione viene mappata sull'arco occupato. Un secondo file, `config/experiment.json`, definisce la flotta, i percorsi (`goal_sequence` per robot), gli scenari di conflitto e il `fault_schedule` — quest'ultimo è anche, letteralmente, la **ground truth** usata per calcolare precision/recall nella valutazione sperimentale (§3).

2.4 Ingestion: Apache Kafka

Un singolo broker Kafka in modalità KRaft (senza Zookeeper, `apache/kafka:3.7.0`) fa da spina dorsale dell'ingestion. Quattro topic principali:

- `telemetry` — un messaggio per tick per robot, partizionato per `robot_id` (3 partizioni): garantisce l'ordinamento per singolo robot mantenendo il parallelismo fra robot diversi.
- `injected_faults` — un record per ogni guasto iniettato (inizio/fine reali, parametri) — la ground truth sperimentale.
- `anomalies` — un evento per ogni anomalia rilevata dal job di detection (salute, deadlock, livelock).
- `fleet_state` — lo stato arricchito di ciascun robot ad ogni tick (telemetria + flag di anomalia), consumato in tempo reale dalla dashboard.

2.5 Generatore sintetico per lo sweep di scalabilità

Gazebo non arriva a volumi di decine di migliaia di messaggi al secondo (il collo di bottiglia è la simulazione fisica, non Kafka/Spark). Per gli esperimenti di scalabilità (§3.2) è stato costruito un generatore puro Python (`generator/synthetic_generator.py` , nessuna dipendenza da ROS) che produce robot-token: percorrono lo stesso grafo del magazzino arco per arco a velocità costante, scegliendo il prossimo nodo **a caso** fra i vicini (nessuna sequenza predisposta, a differenza dei robot ROS), e pubblicano messaggi che rispettano *esattamente* lo stesso schema di `kafka_bridge.py` . Il carico è controllato da due parametri indipendenti (numero di robot-token $\times$ frequenza per robot); una coda a priorità (`heapq`) permette di scalare la schedulazione a migliaia di sorgenti in un solo processo. Il generatore supporta anche l'iniezione di guasti con lo stesso formato e la stessa logica di applicazione di `kafka_bridge.py` , così le metriche di precision/recall si calcolano nello stesso modo sia sui dati reali sia su quelli sintetici.

2.6 Speed layer: detection real-time in Spark Structured Streaming

Il cuore del sistema di detection è `streaming/detection_job.py` : tre query Structured Streaming indipendenti sulla stessa sorgente Kafka (`telemetry`), ciascuna con il proprio sink e il proprio checkpoint — separarle, invece di una query monolitica, mantiene semplice una logica che ha semantiche di trigger radicalmente diverse (per-messaggio contro aggregazioni a finestra con chiavi diverse).

Detection di salute

Trigger a 2 secondi, per ogni messaggio: due tecniche complementari applicate in parallelo.

- **Soglie statiche:** `motor_temp > 55°C` , `motor_current > 2.5A` , `battery_pct < 20%` — deterministiche, calibrabili offline (soglie adattive, §2.7).
- **Isolation Forest** (scikit-learn), applicato via `pandas_udf` sul vettore (`motor_temp` , `motor_current` , `battery_pct` , `v_lin` , `min_obstacle_dist`). Allenato *offline* su un campione sintetico di telemetria nominale generato dagli stessi parametri di `config/experiment.json` (coerenza fra chi genera i dati nominali e chi allena il modello), non online sui dati che sta giudicando — un modello che si auto-allena sui dati che valuta rischierebbe di "abituarsi" ai guasti stessi. Il campionamento sintetico modella esplicitamente la correlazione fra stato di moto e prossimità agli ostacoli (un robot fermo resta "parcheggiato" e la sua distanza da un ostacolo è uno stato stabile per l'intera sosta, non un evento raro — dettaglio emerso e corretto durante l'estensione Passo 14, §2.11).

Ogni messaggio arricchito viene scritto su `fleet_state` (stato live per la dashboard); se una delle due tecniche segnala un'anomalia, viene scritto anche su `anomalies` con `type="salute"` e l'elenco delle ragioni (`threshold_reasons` , eventualmente vuoto se solo l'Isolation Forest ha reagito). Verificato empiricamente che le due tecniche sono complementari, non ridondanti: in un test mirato una soglia su `motor_temp` scattava mentre l'Isolation Forest restava silenzioso (temperatura isolata alta, ma il resto del vettore "sembrava" nominale), e il pattern si è parzialmente invertito su uno spike di corrente.

Rilevamento del deadlock

Finestra scorrevole di 20 secondi (slide 10s) per `current_edge` : se nella finestra compaiono almeno due robot distinti con `task_state="blocked"` sullo stesso arco, viene emessa un'anomalia `type="deadlock"` .

Rilevamento del livelock: un caso di studio sulla semantica dello streaming

Il rilevatore di livelock ha attraversato tre iterazioni successive, ciascuna motivata da un falso positivo reale osservato in dashboard, non da un'analisi teorica preventiva — un esempio istruttivo di quanto le semantiche di windowing di Structured Streaming siano insidiose:

1. **Versione iniziale:** finestra scorrevole di 30s, anomalia se la distanza-sul-grafo dal `goal_node` non cala. La distanza era però agganciata al *nodo più vicino*, non calcolata in modo continuo lungo l'arco: per circa metà del tempo di attraversamento di un arco lungo (fino a 75s a velocità di crociera), la distanza restava perfettamente costante anche con moto reale continuo, generando decine di falsi positivi. **Fix:** distanza calcolata proiettando (x, y) sull'arco corrente e sommando la distanza-sul-grafo (precalcolata con Floyd-Warshall su tutto il grafo, distribuita via `broadcast`) dal punto proiettato al goal.
2. **Secondo bug, indipendente**, trovato dalla suite di test: le query usavano `outputMode("update")`, che riemette una riga ogni volta che l'aggregato di una finestra *cambia*, anche quando la finestra è ancora aperta e parzialmente popolata — un robot osservato nei primi secondi di una nuova finestra (pochi messaggi, poco spostamento visibile *finora*) poteva soddisfare "nessun progresso" su un campione incompleto. **Fix:** `outputMode("append")`, che emette solo dopo che il watermark ha chiuso la finestra per intero (costo: più latenza, fino a un'altra finestra completa).
3. **Terzo bug:** anche con una finestra chiusa per intero, il rilevatore restava una valutazione *a scatto singolo* su 30 secondi — un breve accodamento durante un sorpasso su un corridoio a corsia singola rientra facilmente in una finestra con poco progresso netto, pur essendo un evento normale che si risolve da solo. Un primo tentativo di fix (due aggregazioni a finestra incatenate, per richiedere almeno due finestre candidate consecutive) si è rivelato rompersi su un limite noto di Spark: il watermark del secondo stadio restava bloccato all'epoch nonostante dati reali in arrivo. **Fix definitivo:** stato esplicito per `robot_id` via `applyInPandasWithState` — un "checkpoint" di riferimento (distanza, istante) aggiornato ogni 10s di event time; se il progresso fra un checkpoint e il successivo è sotto soglia si accumula un contatore di stallo continuo, altrimenti si azzerà; l'anomalia scatta solo dopo 60 secondi consecutivi di stallo. Questa versione implementa correttamente la definizione di "livelock" come condizione *prolungata*, non un singolo campionamento sfortunato.

Il rilevatore finale è quindi un vero operatore stateful per chiave, non un'aggregazione a finestra — più pesante computazionalmente (una chiamata Python per robot per micro-batch), ma l'unico che modella correttamente la semantica richiesta.

Infrastruttura Spark

Cluster Spark standalone (master + worker, immagine Docker custom con `pandas / numpy / pyarrow / scikit-learn` aggiunti sopra l'immagine base). Il connettore Kafka (`spark-sql-kafka-0-10`) è risolto a runtime via Maven (`--packages`), con cache Ivy su un volume Docker dedicato. Lo scheduler FIFO di Spark standalone assegna per default tutti i core al primo job avviato: con più job persistenti (detection, query_service, §2.9) è stato necessario fissare esplicitamente `spark.cores.max` per ciascuno.

2.7 Batch layer: persistenza e soglie adattive

`streaming/persistence_job.py` — tre query Structured Streaming indipendenti (stesso pattern del job di detection) che leggono `telemetry`, `anomalies`, `injected_faults` con `startingOffsets=earliest` (a differenza del job di detection, che legge solo `latest` : qui lo scopo è costruire l'archivio storico completo) e li scrivono su **Parquet**, su un volume Docker condiviso montato in sola lettura dal backend. `anomalies` è partizionato per `type`, dato che le query a valle (TAG, valutazione sperimentale) filtrano quasi sempre per tipo.

`offline/adaptive_thresholds.py` chiude un anello di feedback verso la detection: usa `injected_faults` come **ground truth** per classificare ogni anomalia di salute storica come vero o falso positivo, e per i canali con un numero minimo di falsi positivi ricalcola la soglia come il 99.5° percentile (0.5° per la batteria) dei valori osservati nei periodi *nominali* (fuori da ogni finestra di guasto reale) — non un numero scelto a mano, ma calibrato sul rumore nominale realmente osservato. Il job di detection legge queste soglie ricalibrate all'avvio (non un hot-reload a caldo su una query già in esecuzione: il feedback avviene fra una run e la successiva).

2.8 Analisi predittiva offline (time series)

`predictive/forecast_failures.py` stima, per ogni coppia (robot, canale di salute), *quando* il valore supererà la soglia critica del guasto corrispondente — non solo se è già in anomalia, ma con quanto anticipo (*remaining useful life*). La pipeline, in batch su pandas (nessun overhead di una `SparkSession` per questo volume di dati):

1. Ricampiona a bin di 5s la finestra recente di storico (default: ultimi 300s).
2. **Pre-filtro sulla pendenza:** se la variazione per minuto è sotto una soglia minima calibrata sul rumore nominale, si ferma qui — evita fit spurii e falsi allarmi su rumore che imita per caso un trend.
3. Solo in presenza di un trend vero, allena una **regressione lineare ai minimi quadrati** (`numpy.polyfit`, grado 1) e calcola *analiticamente* l'istante di incrocio con la soglia critica ($t = (\text{soglia} - \text{intercetta}) / \text{pendenza}$) — nessun bisogno di generare un forecast punto-per-punto.
4. Le soglie critiche coincidono deliberatamente con i valori-obiettivo delle firme di guasto (§2.2): 85°C è il `plateau_temp_c` di `deriva_termica`, 4.5A è il `peak_a` di `spike_corrente`.

La scelta della tecnica è stata rivista in corso d'opera: la prima versione usava **ARIMA** (`statsmodels`), come indicato dal piano originale. Un problema concreto (il default `trend='n'` di `ARIMA(p,1,0)` fa convergere il forecast a un livello asintotico costante invece di continuare a estrapolare una rampa) e la constatazione che il segnale è quasi lineare per costruzione (le firme di guasto sono rampe lineari verso un plateau) hanno motivato il passaggio a una regressione lineare — tecnica più semplice, più facile da verificare, con risultati praticamente identici ad ARIMA sullo stesso segnale reale (130.5s contro 130.0s di lead time misurato).

2.9 Serving layer: Spark SQL e sistema TAG con Hugging Face

Il quarto e ultimo requisito tecnologico obbligatorio — un LLM — è implementato come layer **TAG (Table-Augmented Generation)**: l'utente pone una domanda in linguaggio naturale, il sistema la traduce in SQL, la esegue sui dati storici Parquet, e restituisce il risultato.

Esecuzione SQL: Spark SQL, non un motore embedded

Il piano originale prevedeva DuckDB (libreria Node embeddable). È stata preferita una soluzione che riusa il cluster Spark già in piedi per la detection e la persistenza: `streaming/query_service.py`, un processo Spark **persistente** con un server HTTP nativo (`http.server`, nessuna dipendenza aggiuntiva per due sole route) che ricrea le temp view sulle quattro cartelle Parquet ad ogni richiesta (economico: è solo discovery dello schema, non un'azione, quindi vede sempre i dati più recenti scritti dal batch layer) ed esegue la query con un `LIMIT` di sicurezza. Evitare `spark-submit` per ogni domanda risparmia i circa 10 secondi di avvio JVM per query, inaccettabili per un endpoint interattivo.

Generazione SQL: Qwen2.5-Coder-32B via router Hugging Face

Il modello — `Qwen/Qwen2.5-Coder-32B-Instruct` — non è ospitato in locale: viene chiamato tramite il router **Inference Providers** di Hugging Face (`https://router.huggingface.co/v1/chat/completions`, API compatibile OpenAI, autenticazione Bearer). Questa scelta evita di dover gestire GPU/quantizzazione/`nvidia-container-toolkit` nel container per un modello da 32 miliardi di parametri, mantenendo comunque un LLM realmente open-source (non un servizio proprietario chiuso).

Pipeline dell'endpoint `POST /api/tag`

Backend Node.js, quattro moduli con responsabilità separate:

- `promptBuilder.js` — costruisce il prompt: schema completo delle quattro tabelle Parquet (con le peculiarità del dialetto Spark SQL rilevanti: accesso a STRUCT con dot notation, `array_contains` per campi ARRAY, `timestamp_millis` per i timestamp epoch), 5 esempi few-shot, regole esplicite (solo `SELECT`, nessuna spiegazione testuale, `LIMIT` ragionevole se non specificato dalla domanda).
- `LlmService.js` — client verso il router Hugging Face.
- `sqlGuard.js` — *difesa in profondità*: ripulisce eventuali blocchi markdown residui e valida che la query generata sia una singola `SELECT / WITH`, priva di parole chiave di scrittura (con controllo a confini di parola, per non bloccare per errore identificatori come `created_at` a causa della sottostringa "creat"), **sia lato Node sia lato `query_service.py`** — due controlli indipendenti sullo stesso vincolo.
- `tagService.js` — orchestra prompt → LLM → guardia → esecuzione, con **un retry** automatico (ri-prompting il modello con l'errore ricevuto) se la guardia rifiuta la query o l'esecuzione fallisce.

Verificato che il modello generalizza oltre gli esempi few-shot forniti: nessuno dei 5 esempi nel prompt usa una `JOIN` o più di una CTE, eppure il sistema ha generato correttamente una query con CTE e `JOIN` per una domanda che lo richiedeva (§3.1).

2.10 Presentazione: backend Node.js e dashboard

Il backend (Node.js/Express) consuma `fleet_state` da Kafka (client `kafkajs`) e fa push via WebSocket ai client connessi; espone inoltre le route REST per il layer TAG, le previsioni, il controllo del generatore sintetico, il controllo della flotta reale e i risultati sperimentali. La dashboard (HTML + canvas 2D, JavaScript puro, nessun framework) disegna la mappa del magazzino, i robot colorati per stato con un anello di segnalazione per le anomalie, gli archi a capacità singola tratteggiati. Per un'animazione fluida nonostante il micro-batch di detection arrivi al massimo ogni 2 secondi, il client applica *dead reckoning*: estrapola la

posizione ad ogni frame usando la velocità nota dall'ultimo messaggio, invece di attendere il prossimo pacchetto.

Fra i pannelli disponibili: stato di tutti i robot (reali e sintetici) in tabella, log degli eventi comportamentali (deadlock/livelock) in tempo reale, pannello previsioni (dal layer predittivo), casella di interrogazione in linguaggio naturale, pannello di controllo per avviare/fermare il generatore sintetico scegliendo topologia e guasti, e — nell'estensione più recente — un pannello che lancia gli esperimenti di valutazione sperimentale *on demand* mostrandone i risultati mano a mano che ciascun sotto-esperimento termina, invece di un refresh periodico su grafici pre-generati.

2.11 Estensione: self healing sulla flotta reale

Oltre ai 13 passi del piano originale, su richiesta esplicita per una dimostrazione dal vivo, è stato realizzato un primo anello di retroazione reale limitato alla flotta ROS/Gazebo. Fino a questo punto il sistema era puramente diagnostico (rilevare, prevedere, segnalare — mai agire su ROS), coerentemente con il principio "il robot è solo la sorgente dati". L'estensione introduce un'eccezione deliberata e documentata, non una deriva silenziosa dello scope.

- **Corridoi fisici:** i due archi a corsia singola del grafo sono stati fisicamente ristretti in Gazebo con pareti statiche, rendendo osservabili su dati reali (non solo sintetici) sia il deadlock sia il livelock.
- **Iniezione di guasti dal vivo:** un nuovo canale di controllo ROS (`~fault_inject`, JSON su `std_msgs/String`) permette di aggiungere un guasto al bridge *a runtime* — riusa esattamente la stessa logica di attivazione/applicazione/logging dei guasti pre-schedulati, nessuna scorciatoia: il guasto iniettato dal vivo attraversa il topic `telemetry` reale.
- **Robot di riserva dedicati** (R4, e R8 nello scenario esteso): fermi finché non servono, mai scelti fra i robot già in missione — garantisce un sostituto sempre pronto.
- **Anello automatico:** quando un'anomalia di salute su *soglia fissa* (deterministica) viene rilevata su un robot reale, il backend invia il robot colpito verso un nodo di riparazione e spaccia una riserva disponibile sulla missione originale (letta da `config/experiment.json`) — ripartendo da capo, non dal punto esatto di interruzione (limite dichiarato, §4.2).

Nota tecnica di rilievo — falsi positivi statistici collegati a un'azione reale. La prima versione dell'anello automatico si agganciava al flag `health_anomaly` di `fleet_state`, che è l'OR fra soglie fisse e Isolation Forest. Con 8 robot attivi (scenario esteso), il tasso di falsi positivi *strutturale* dell'Isolation Forest (parametro `contamination=0.02`: per definizione, circa il 2% delle predizioni scatta sempre, anche su dati perfettamente nominali) è diventato consequenziale — quasi tutta la flotta finiva "in riparazione" entro un minuto senza alcun guasto iniettato. La correzione risolutiva non è stata continuare a tarare il modello statistico, ma **cambiare quale segnale aziona l'azione reale**: l'anello automatico ora ascolta solo le anomalie di salute su soglia fissa (deterministiche per costruzione), lasciando l'Isolation Forest attivo altrove nel sistema per il suo valore di segnale "morbido". È una lezione generale, non specifica di questo progetto: un modello statistico con un tasso di falsi positivi noto per costruzione può essere accettabile come segnale informativo passivo, ma richiede cautela esplicita quando viene collegato per la prima volta a un'azione reale.

2.12 Orchestrazione e deployment

L'intero sistema è orchestrato con Docker Compose: un container Kafka, due container Spark (master/worker), un container Node per backend e dashboard, un container ROS (basato su `osrf/ros:noetic-desktop-full` con i pacchetti TurtleBot3, noVNC per l'accesso GUI in debug). All'interno del container ROS, un processo `supervisord` gestisce in parallelo: `roscore`, la simulazione Gazebo multi-robot, e diversi micro-servizi HTTP di controllo con lo stesso pattern architetturale (server `http.server` nativo, senza framework, per poche route interne): il generatore sintetico, il controllo della flotta reale (guasti live, missioni), e il lancio on-demand degli esperimenti di valutazione. L'intero stack si avvia con un singolo `docker compose up -d`, senza comandi manuali post-avvio.

3. Risultati

3.1 Effectiveness: precisione della detection, della previsione e del TAG

Gli esperimenti di effectiveness usano `injected_faults` come ground truth oggettiva: un'anomalia rilevata è un vero positivo se cade all'interno della finestra temporale di un guasto realmente iniettato per quel robot, falso positivo altrimenti.

Metrica	Valore	Dettaglio
Detection salute — precision	1.00	8 robot sintetici, 3 con guasto noto (<code>spike_corrente</code>), 5 senza; 0 falsi positivi
Detection salute — recall	1.00	0 falsi negativi
Detection salute — F1	1.00	TP=3, FP=0, FN=0, TN=5
Previsione — errore medio assoluto (MAE)	1.6 s	su lead time previsti di centinaia di secondi (3 scenari sintetici a pendenza nota)
TAG — execution accuracy	21/22 (95%)	22 domande di riferimento in linguaggio naturale, confronto con SQL di verità scritto a mano

L'unica domanda TAG "sbagliata" ("qual è la temperatura media dei robot con guasti") si è rivelata un'**ambiguità genuina della domanda**, non un errore del sistema: la query di riferimento calcola la media su tutta la telemetria dei robot coinvolti, mentre il modello ha scelto — in modo altrettanto legittimo — di calcolarla solo sulle anomalie di temperatura rilevate. È stata lasciata così come esempio concreto dei limiti intrinseci dell'interrogazione in linguaggio naturale, invece di correggere la domanda per farla "tornare" artificialmente.

Limite metodologico dichiarato. Le metriche di detection sopra sono misurate sul generatore sintetico (Passo 12), non sulla flotta ROS/Gazebo reale: la ground truth richiede un numero di robot e di ripetizioni controllate che sarebbe troppo lento/costoso da ottenere avviando ripetutamente l'intera simulazione fisica. È un compromesso dichiarato, non nascosto: la pipeline di detection è identica sui due percorsi (stesso schema, stesso codice Spark), e la fedeltà fisica della sorgente ROS/Gazebo è stata comunque verificata qualitativamente e in modo end-to-end (§3.3), ma le cifre di precision/recall riportate qui non provano che la detection funzioni altrettanto bene sul rumore realistico della simulazione fisica quanto su quello, più pulito, del generatore sintetico.

3.2 Efficiency: scalabilità e latenza

Metrica	Valore	Dettaglio
Throughput — scaling pulito fino a	20 000 msg/s	100% del target raggiunto
Throughput — punto di rottura	~40 000 msg/s target	raggiunto 30 854 msg/s, 77% del target; degrado ulteriore a target più alti
Latenza onset → alert	media 30.2 s (3/5 prove riuscite)	misura volutamente prudente/larga, vedi nota sotto

Il collo di bottiglia individuato per il throughput è il processo Python singolo del generatore sintetico (non Kafka o Spark, che nei test non mostrano saturazione alla stessa scala). Il numero di "~30s" di latenza onset → alert **non è la vera latenza minima del sistema** (osservata altrove, in condizioni dedicate, dell'ordine di pochi secondi, coerente col trigger di 2s della query di salute): la tecnica di misura usata negli esperimenti (due consumer Kafka interrogati in sequenza, non in parallelo) misura "quanto tempo impiega lo script di test ad accorgersene", non "quanto tempo impiega il sistema a produrre l'alert" — un limite dichiarato del metodo di misura, non delle prestazioni reali.

3.3 Verifica end-to-end sulla flotta reale

Oltre alle metriche quantitative sopra (misurate sul generatore per motivi di scala, §3.1), il comportamento end-to-end è stato verificato con dati realmente prodotti dalla simulazione fisica ROS/Gazebo, non solo letto dal codice:

- **Conflitto fisico reale:** due robot comandati verso l'estremità opposta dello stesso corridoio a corsia singola nello stesso momento hanno prodotto uno stallo reale (`task_state="blocked"` osservato su Kafka, non simulato) — prima conferma empirica che i choke point del grafo hanno un effetto fisico reale, non solo un vincolo dichiarato in un file di configurazione.
- **Iniezione di un guasto dal vivo** su un robot reale, tramite la dashboard: visibile nella telemetria reale (`motor_current` salito ben oltre il valore nominale) sullo stesso topic `telemetry` usato per tutto il resto della pipeline.
- **Anello di self healing:** l'anomalia di salute su soglia fissa, rilevata su dati reali, ha correttamente inviato il robot colpito verso il nodo di riparazione e dispacciato una riserva disponibile sulla missione originale — con selezione corretta fra più riserve disponibili quando un secondo guasto (pre-schedulato, non iniettato manualmente) è scattato in concomitanza sullo scenario a 8 robot.
- **Ripristino manuale:** il comando "rimetti in servizio" da dashboard riporta correttamente il robot riparato verso il nodo di riserva.

3.4 Copertura dei requisiti del corso

Requisito (Topic 7)	Dove è coperto
Ingest di stream continui da sensori	Telemetria TurtleBot3 → nodo-ponte ROS → Kafka
Real-time analytics	Detection in Spark Structured Streaming (salute, deadlock, livelock)
Offline analytics su storico → analisi predittiva	Previsione time-series dei guasti (regressione lineare)
Kafka / Spark Streaming	Ingestion e Structured Streaming, presenti end-to-end
Prediction algorithms su time series	Regressione lineare su trend dei canali di salute (ARIMA valutato e poi semplificato)
LLM (open source)	TAG text-to-SQL con Qwen2.5-Coder-32B via router Hugging Face
Esperimenti: effectiveness + efficiency	Suite di valutazione dedicata, con ground truth oggettiva

4. Osservazioni conclusive

4.1 Lezioni tecniche

1. **Soglie fisse e modelli statistici sono complementari, non intercambiabili.** Nei test mirati del Passo 7, ciascuna tecnica ha rilevato casi che l'altra non copriva. La combinazione è più robusta di scegliere una delle due a priori.
2. **Le semantiche di windowing di Structured Streaming sono sottili e vanno verificate empiricamente, non assunte.** Il rilevatore di livelock ha richiesto tre iterazioni (proiezione continua sull'arco, `outputMode("append")` invece di `"update"`, infine uno stato esplicito per chiave via `applyInPandasWithState`) prima di implementare correttamente la definizione di "stallo prolungato" — un'aggregazione a finestra singola, per quanto ben calibrata, non può esprimere una condizione che deve persistere nel tempo attraverso più finestre.
3. **Un modello con un tasso di falsi positivi strutturale diventa consequenziale solo quando viene collegato a un'azione reale.** L'Isolation Forest (`contamination=0.02`) è stato tranquillamente in produzione come segnale passivo per l'intero progetto; il problema è emerso solo dopo averlo (indirettamente) collegato a un'azione automatica su hardware reale, ed è stato risolto a livello architetturale (cambiare il segnale sorgente), non tarando ulteriormente il modello.
4. **La fedeltà dei dati sintetici di training conta quanto la scelta del modello.** Un campionamento indipendente di feature correlate nella realtà (velocità e distanza da ostacoli, per un robot fermo) ha sotto-rappresentato una combinazione tutt'altro che rara, contribuendo ai falsi positivi discussi sopra.
5. **Un modello più semplice, quando il segnale lo giustifica, non è un compromesso.** La regressione lineare ha prodotto risultati praticamente identici ad ARIMA sullo stesso segnale reale (differenza di 0.5s su un lead time di oltre due minuti), con una tecnica enormemente più semplice da spiegare, verificare e mantenere.
6. **NaN è un float Python legale, ma non è JSON valido.** Una metrica che può assumere legittimamente il valore NaN (es. precisione con $TP+FP=0$) va sempre sanificata esplicitamente prima della serializzazione: Python la accetta senza errori, ma qualunque consumatore JSON standard (incluso il parser nativo dei browser) fallisce silenziosamente su un token `NaN` letterale nel corpo della risposta.

4.2 Limiti dichiarati

- Le metriche ufficiali di detection (§3.1) sono misurate sul generatore sintetico, non sulla flotta ROS/Gazebo reale, per ragioni di scala/tempo sperimentale — limite dichiarato esplicitamente, non nascosto.
- Nell'anello di self healing, la riserva riparte la missione del robot guasto **da capo**, non dal punto esatto di interruzione.
- Non esiste un secondo livello di riserva: se una riserva stessa si guasta durante una missione, non viene sostituita.

- La larghezza dei corridoi fisici (0.8 m) è stata tarata empiricamente, non validata in modo esaustivo su tutte le combinazioni di traiettorie possibili.
- La soglia di conferma del livelock (60s) è una costante fissa, non adattiva al ritmo operativo della flotta.
- La misura di latenza onset → alert riportata (§3.2) è volutamente prudente per un limite del metodo di misura (consumer interrogati in sequenza), non la vera latenza minima del sistema.

4.3 Sviluppi futuri

- Tracciare l'avanzamento esatto della missione interrotta, per permettere a una riserva di riprendere da dove il robot guasto si era fermato invece di ripartire da capo.
- Un secondo livello di riserva, per tollerare il guasto di una riserva già dispacciata.
- Calibrazione delle soglie adattive per singolo robot, non solo a livello di flotta, se il volume di storico per robot crescesse a sufficienza.
- Una misura di latenza onset → alert più rigorosa, con consumo parallelo (non sequenziale) dei topic coinvolti.
- Ripetere la misura di precision/recall della detection direttamente sulla flotta ROS/Gazebo reale, per rimuovere il limite metodologico dichiarato in §3.1/§4.2.
- Un token Hugging Face dedicato al progetto, in sostituzione di quello di sviluppo attualmente riusato da un altro progetto dell'autore.

Relazione generata a partire dalla documentazione tecnica di progetto (`CLAUDE.md` , `PLAN.md` , `docs/passi/*.md`), redatta passo per passo durante lo sviluppo e verificata empiricamente ad ogni fase.